Let It Go: Relieving Garbage Collection Pain for Latency Critical Applications in Golang

Junxian Zhao
jzhao@uccs.edu
University of Colorado
Colorado Springs

Xiaobo Zhou*
waynexzhou@um.edu.mo
IOTSC, University of Macau
Macau S.A.R

Sang-Yoon Chang
schang2@uccs.edu
University of Colorado
Colorado Springs

Chengzhong Xu
czxu@um.edu.mo
IOTSC, University of Macau
Macau S.A.R

ABSTRACT

Garbage Collection (GC) is a representative automatic memory manager widely deployed in popular programming languages, such as Java, C#, and Golang (Go). Through GC, these languages provide programmers with flexibility and safety. However, GC leads to non-trivial overhead in compute and memory resources during application runtime. GC threads compete with non-GC threads (mutators) of an application, which particularly impacts latency-critical (LC) applications and causes long tail latency. Existing GC approaches do not efficiently address the interference, as GC is triggered passively without a global insight of the application; or they employ incremental GC to reduce the interference, while the incremental progress is not dynamically tailored during GC process according to runtime characteristics, which leads to significant performance degradation upon bursty requests.

We present LEGO, an efficient and non-intrusive GC framework that deploys a novel elastic incremental GC mechanism integrated with an adaptive GC scheduler to reduce CPU contention between GC and mutators, and improve resource utilization and QoS of LC applications. We choose to develop LEGO in Go, as Go is specifically designed for cloud applications and the current GC mechanisms for JVM are ineffective for Go due to its unique GPM thread scheduling and memory allocation model. LEGO adapts incremental GC into Go to tackle the full GC issue and addresses resource contention with a proactive scheduler for adaptive GC triggering. Importantly, LEGO leverages the elastic incremental GC mechanism in mitigating the interference from unavoidable GC in face of bursts of requests. We implement and evaluate LEGO with popular LC applications developed in Go. Results show that compared to the default Go GC and a tailored G1 GC, LEGO significantly improves both the tail latency and throughput for LC applications.

*The corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HPDC '23, June 16–23, 2023, Orlando, FL, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0155-9/23/06…$15.00
https://doi.org/10.1145/3588195.3592998

CCS CONCEPTS

• **Software and its engineering → Garbage collection**; • **Computer systems organization → Cloud computing**.

KEYWORDS

Golang (Go), Garbage Collection (GC), GC Scheduler, Latency Critical (LC), Tail Latency

ACM Reference Format:
Junxian Zhao, Xiaobo Zhou, Sang-Yoon Chang, and Chengzhong Xu. 2023. Let It Go: Relieving Garbage Collection Pain for Latency Critical Applications in Golang. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23), June 16–23, 2023, Orlando, FL, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3588195.3592998

1 INTRODUCTION

Cloud computing has achieved remarkable success over the last decade due to features including resource sharing, felxibility, and scalability. Cloud systems also require fast and safe concurrency provided by programming languages. This is where Golang (a.k.a Go) comes for, which is also known as *Language of Cloud*. Designed for high concurrency, Go emphasizes concurrent programming for cloud systems. It is being used to build popular cloud applications [5–7, 41] at a massive scale.

As many modern programming languages, Go is an auto-memory management language. Instead of explicitly removing obsolete objects, it deploys GC for memory management. GC is executed automatically in background to reclaim memory for the runtime heap. During GC runtime, it may pause the entire application, a phenomenon known as Stop The World (STW) [18] in other languages such as Java.

Motivation. Go's unique GPM model is designed to improve thread-switching performance. Different from the state-of-the-art GCs, such as G1 GC [24], ZGC [20] and Shenandoah GC [19] for the Java Virtual Machine (JVM), Go employs a concurrent, non-generational, and non-incremental GC. This corresponds to less STW overhead, a runtime heap layout without distinction between young and old objects, and the ability to run GC on the partial memory heap. Go also leverages the TCmalloc memory allocator [26], which allocates objects at the thread level, whereas JVM allocates objects from a universal heap when the thread-level stack buffer is full. Furthermore, Go GC is non-copy-and-compact. These features

make existing GC optimizations for JVM not suitable or inefficient for Go.

We find Go applications often experience significant latency, particularly tail latency in face of CPU interference and bursty requests. GC threads interfere with non-GC threads (mutators), which results in performance degradation of latency-critical (LC) applications. Our experiments show that GC interference on LC applications such as key-value stores developed in Go can cause up to 4× increase of the 99th percentile (p99) query latency compared to when the key-value stores are running without GC. The tail latency becomes more severe when the input requests are bursty. Even worse, our experiments find that GC is more likely (more than 2×) to be triggered in face of bursty requests.

Therefore, Go demands for a new GC mechanism and a scheduler that take into account interference of resource usage introduced by mutators of the concurrent GC. Specifically, Go GC should satisfy the following requirements for maintaining service level objective (SLO) of LC applications.

- LC applications should maintain low tail latency for meeting its SLO objectives.
- The new GC should reduce its interference on mutators of the application, especially upon bursty requests.
- The new GC needs to be transparent and generally applicable to all applications developed in Go. Thus, the source code of applications need not be modified.

Limitation of state-of-the-art approaches. There are work in memory management focusing on GC optimization, mostly in JVM. Previous works [32, 37, 38, 51] introduced new algorithms and modified heap layout to reduce GC overhead. Recent works [28, 50] propose to schedule with fine-grained hardware resources, i.e. CPU and memory, for different threads in GC. There are also studies [44, 45] that focus on GC on far-memory and resource disaggregated systems. However, due to the semantic gap between JVM and Go designs, most of the existing optimizations are not available or directly applicable for Go. Importantly, the existing works lack runtime adaptivity for incremental GC design.

Challenge I - pre-fixed incremental volume. The production G1 GC deploys an incremental GC mechanism for JVM, where the GC pause time and reclaimed memory volume are pre-fixed when an application is started up or during a non-GC period. However, G1 GC cannot elastically decide the volume of the current GC during GC runtime. In face of bursty requests, GC activities dramatically increase the latency of an application violating its SLO. This is unavoidable in the design of the current incremental GC mechanism.

Challenge II - passive GC triggering. The state-of-the-art GCs (such as G1 GC, ZGC and Shenandoah GC) only trigger GC progress early enough to avoid Out Of Memory (OOM) errors. They do not consider the resource usage of GC progress at runtime. The default GC in Go limits the total CPU time usage of GC progress to a 30% threshold, which does not delicately match the runtime CPU usage. GC progress tends to compete CPU resources with mutators dynamically. However, the Go scheduler triggers GC even when the CPU usage is higher than 70%, which can lead to severe performance degradation in latency-critical applications.

Challenge III - integration into Go. The existing GC optimizations are difficult to be directly transplanted into Go, as Go employs a different memory layout, i.e., GPM model, and it uses TCmalloc and non-copy-and-compact GC. Currently, the default Go GC only supports full GC.

Key insights and contributions. In this paper, we present an efficient, non-intrusive GC scheduling framework, LEGO that deploys a novel elastic incremental GC mechanism integrated with an adaptive GC scheduler to reduce CPU contention between GC and mutators, and improve resource utilization and QoS of LC applications developed in Go.

To address the first two challenges (pre-fixed incremental volume and passive GC triggering), LEGO designs a unique adaptive incremental GC mechanism that proactively triggers GC and dynamically tailors the incremental progress during GC according to runtime characteristics. Specifically, it obtains GC processing volume at runtime in face of detecting a burst of requests, and proactively triggers and adjusts the GC processing volume based on the available resources. To facilitate its adaptive incremental GC, LEGO designs Quad-Color marking algorithm for dynamically GC triggering.

To address the third challenge, LEGO adds new data structures (write barrier and P&G-level remember set) for recording object reference modifications and Goroutine migration during GC runtime. Furthermore, it adds a P-level reference counter to reduce the overhead of Goroutine migration and maintain the consistency of object reference during GC.

This paper makes the following contributions.

- We conduct in-depth characterization and analysis of Go GC interference on LC applications, especially its performance impact upon bursty requests. We apply a statistic approach that leverages on the obtained runtime characteristics for diagnosing GC interference.
- We design an elastic incremental GC mechanism with a proactive GC scheduler. Furthermore, we develop a Quad-Color marking algorithm for dynamically triggering the incremental GC mechanism.
- We implement LEGO in Go runtime. It is transparent to applications and the OS. LEGO enables flexible and robust GC, which can be deployed in any mark-and-sweep GC system including JVM GC.

We evaluate LEGO with popular Go-developed cloud applications (Badger, Bolt, Rosedb, and TiDB) and benchmark workloads. Experiment results show that LEGO can reduce the average 99th percentile latency up to 35.6% and 42.7%, and improve the system throughput by 33% and 42% compared to the default Go GC and a tailored G1 GC, respectively. LEGO software artifact is available at https://github.com/lotusknight/LEGO.

Limitations of the proposed approach. LEGO considers LC applications that have bursty requests. It reschedules GC with transiently available system resources. It is possible though uncommon that an LC application receives consistent high volume of traffic. In this scenario, GC may be triggered frequently or even continuously. LEGO would be less or ineffective compared with the default Go GC. Note LEGO can be transiently switched to the default Go GC.

Currently, LEGO supports Go programs. Its principle and design can be applied to any programming language runtimes including JVM that employ automatic memory management.

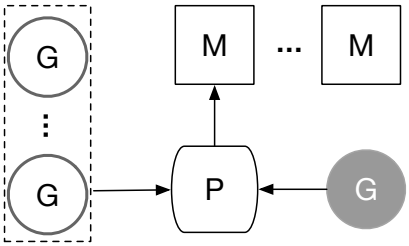


Figure 1: Go GPM abstract model.

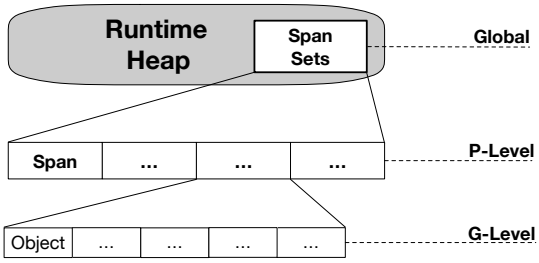


Figure 2: Go heap layout and memory management.

In the rest of the paper, Section 2 presents background and motivational studies. Sections 3 and 4 describe LEGO design and implementation. LEGO is evaluated in Section 5. We discuss related work in Section 6 and conclude in Section 7.

2 BACKGROUND AND MOTIVATION

2.1 Go Thread Scheduling and Heap Layout

Thread scheduling. Go introduces a unique GPM model as abstracts for flexible and low-overhead runtime thread management and concurrent programming. Figure 1 shows the abstract model. G (Goroutine) is the basic execution unit in Go, which is detached from an OS-level thread to reduce thread switching overhead. Different Gs run in parallel during runtime. Similar to a process in Linux, each G has execution states such as *runnable* and *running*. The solid gray G represents the current running Goroutine, while a white blank G is not being executed by any thread. Multiple Ms represent OS-level threads. Each P (processor) must be attached to a M (machine). Go runtime can reschedule a P to a different M. Each P has a local queue for *runnable* Gs. By running on an M, P executes Gs from its *runnable* queue. Compared with runtime thread switching based on system calls, G switching is more flexible as P reuses the same OS thread (M) for running different Gs. With the GPM model, Go reduces overhead in creating and scheduling threads.

Heap layout and memory management. Figure 2 shows the process of heap layout and memory management. Data allocation at runtime happens on the G stack and runtime heap. If data on the stack is shared by multiple Gs, it will be placed on the heap. Go deploys TCmalloc allocator to pre-allocate memory from heap for each P. Each P has its own cache, *mcache*, for different sizes of objects. Each allocation size set is called *mspan*. When a G performs memory allocation, it actually operates on the corresponding *mspan* from the current P. P requests more *mspan* from a set of *mspan* on *mcentral*. A *mcentral* is a a data structure that holds a set of free memory blocks with different sizes. It is used for initializing a set of *mspan* with different sizes from the runtime heap.

Table 1: Time spent on each GC phase (ms).

Sweep Termination		Concurrent Mark		Mark Termination	
A	B	A	B	A	B
0.09	0.09	28	0.58	0.005	0.005
0.10	0.10	27	0.58	0.008	0.008
0.11	0.10	32	0.57	0.017	0.017
0.13	0.12	23	0.60	0.011	0.01

2.2 Garbage Collection (GC)

Go GC semantics. Fig 3 shows four phases in Go GC, 1) Sweep termination (STW), 2) Mark phase (GCmark), 3) Mark termination (STW, GCmarktermination), and 4) Sweep phase (GCoff). In each GC cycle (phases 1 to 3), Go scans the whole heap and marks alive objects. Two phases (STWs and Concurrent Marking) are executed during this period. Concurrent Marking time is much longer (>100X) than STW time. During the Concurrent Marking phase, mutators are running in parallel with GC processes. Each allocation by a mutator is followed by a number of scan operations, *AllocateAssist*. After a complete GC cycle, in phase 4 Go sweeps and reallocates obsolete objects when a new object is allocated, which is also concurrent. Note Go does not deploy a generational GC mechanism, instead, the objects in runtime heap are non-movable. They are allocated in their final positions, which is different from the traditional GCs in JVM. Thus, JVM GCs cannot be directly transplanted to Go since they have different basic GC logics, i.e. JVM adopts copy-and-compact GC.

Tri-color marking. All tracing-based GCs use tri-color marking algorithm [10]. It traces reachable objects from RootSets. When the marking phase starts, it puts all objects in RootSets to a list which grays them. GC worker threads fetch objects from the list and trace pointers from the objects. Objects with untraced pointers are grayed and put into the list recursively. Other reachable objects are marked black. Three colors, white, gray and black, represent different states of marking: 1) a white object is not currently reachable, 2), a black object is reachable and all its pointers are traced, 3) a gray object is marked, while it has pointers not exhausted by scan worker threads. Gray objects are put in a work list for further tracing. When the list of gray objects is empty, the Concurrent Marking phase is finished. The white objects left are unreachable and can be swept in the later phase.

2.3 GC Characterization and Analysis

CPU interference. Discord developer logs give the CPU usage and response latency from a Discord chatting server written in Go [1]. There are peaks of both CPU usage and response time every 2 minutes when Go GC is triggered. By testing with different configurations, it turns out that concurrent Marking of Go GC costs significant hardware resources and leads to interference on mutators.

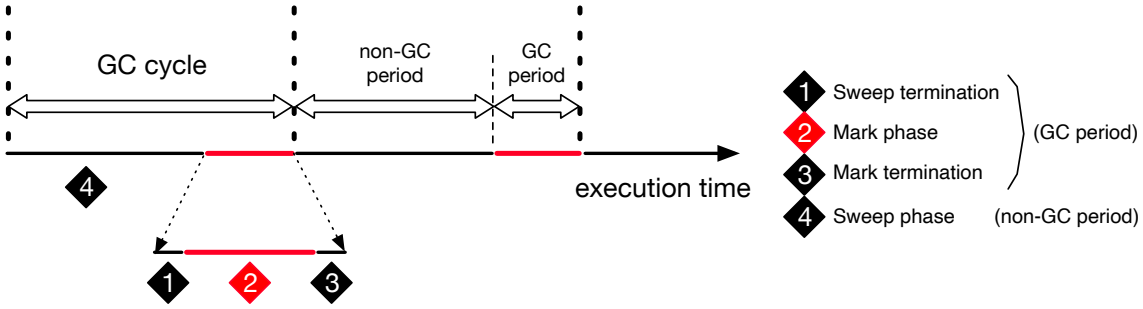


Figure 3: Go GC cycles with four phases during execution time.

Reference impact. We build two microbenchmarks (A and B) using Go. Microbenchmark A stores a map with key (integer) and value (a reference to structure). The time spent on four GC cycles are 28.095ms, 27.108ms, 32.127ms, and 23.141 ms, respectively. For each cycle, we show the time elapse of each phase in Table 1. Obviously, the concurrent marking phase dominates the overall GC overhead. Microbenchmark B changes the value to integer. Go GC ignores scalar(int, float) when tracing references during marking. The time spent on 4 GC cycles are 0.675ms, 0.688ms, 0.687ms, and 0.73ms, respectively, much smaller than those in A. We can observe that the overhead of concurrent marking is severely impacted by the number of references. To mitigate the overhead in Goroutine immigration, we reduce the amount of metadata for recording references between different Goroutines by introducing a P-level bit mask technique.

Memory demand and usage. Both CPU usage and memory demand of an application fluctuate at runtime. Here, we use Poisson distribution to generate random input volume. Case studies are executed on Badger [4], a key-value (KV) store written in Go. Figure 4 shows the input volume and total memory usage per second. The vertical green lines show that the memory usage drops sharply each time GC is triggered, because only little data are reused for the KV store. The memory demand in each second consists of the basic memory usage of the KV store (about 200MB) and the input volume, which is pretty low compared to the total memory usage. Though the default Go GC triggering avoids OOM errors, it does not consider CPU interference when GC is triggered, which often causes high tail latency for LC applications. As the memory demand fluctuates from time to time, it is unnecessary to trigger GC as late as possible. The memory demand is not always beyond a high threshold, which allows us to trigger GC dynamically as the full GC is not required. We leverage the opportunities to proactively trigger GC and thus avoid CPU interference in face of a burst of requests.

2.4 Case Studies

We consider two different settings of LC application Badger, 1) input stable requests at the server maximum capacity $5 * 10^4$ request per second (RPS), and 2) input bursty requests following a Poisson distribution. In each interval (one second), Badger sends n requests where n obeys a Poisson distribution ($\lambda = 5 * 10^4$). We use the second setup to emulate a real-world scenario. The server setting is identical to that in Section 5.

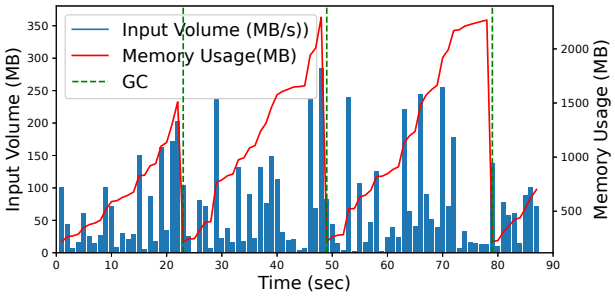


Figure 4: Memory demand and actual usage for Badger with Random input volume. Three sharp declines are due to GC Triggering. Heap size is set to 2,400 MB.

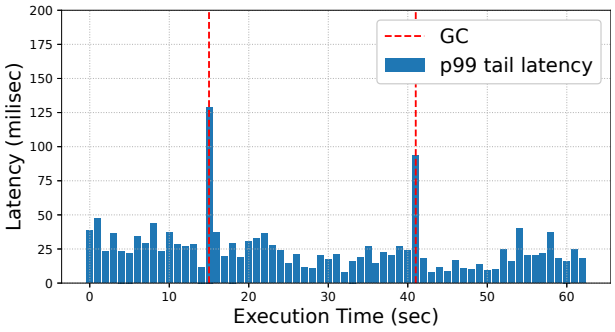


Figure 5: P99 tail latency under static input volume.

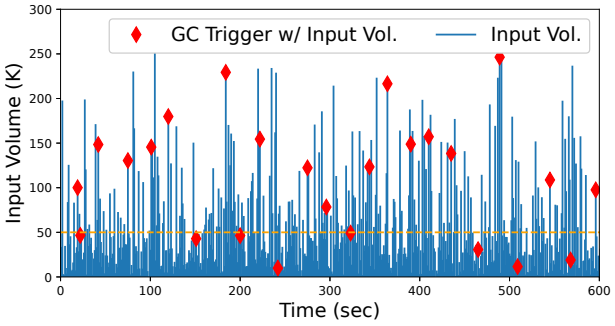


Figure 6: GC triggering affects the input volume.

Table 2: Statistics of GC trigger timing with input volume.

	Input Volume <50,000	Input Volume >=50,000
GC #	67(29.9%)	157(70.1%)
Input #	3893(64.2%)	2167(35.8%)

Figure 5 depicts the $99th$ tail latency due to setting 1. Each time Go GC is triggered, the tail latency increases to more than 4X of the average p99 tail latency. As Go GC runs concurrently with mutators, it competes CPU resources and prolongs the application response. While the average CPU usage is only around 40% without GC, the CPU usage is more than 90% upon GC triggering. Though the GC pause and concurrent marking time are pretty small, the overall overhead of GC in the tail latency is nontrivial.

After learning the impact of CPU interference due to GC, we examine how the input volume affects GC trigger timing in setting 2. We run the case study for 100 minutes and record the real-time RPS and timing of GC triggering. Figure 6 shows the results in random 10 minutes. The y-axis of blue lines shows the RPS value when GCs are triggered (red darts). The orange dash line (y=50K) represents the server maximum capacity. If the RPS is higher than 50K, the LC application cannot reach its SLO due to the bursty traffic condition. We can clearly see that most of red darts are scattered above the orange dashed line. We collect the data of the whole 100 minutes experiment and show the statistics in Table 2.

Table 2 shows that more than 70% of GCs are triggered upon large input volumes, while large input volumes only occupy 36% of all input volumes. The results show that GC tends to be triggered when a burst of requests arrives.

We examine the interesting phenomenon and reveal that the current Go GC use a heap usage increment ratio mechanism to predict how early enough does it trigger GC to avoid OOM errors. Thus, a burst of requests represents a higher rate of heap usage increment ratio, which in return results in a higher probability of reaching the GC threshold since a large number of objects are allocated in the heap. As a result, more requests are involved when GC is running during a burst of requests, which leads to more significant tail latency.

The state-of-the-art GCs, including both Go GC and JVM GCs, are triggered passively, whether in a fixed time interval or in a predictive way to avoid OOM errors. Such a GC triggering mechanism is unable to address the high tail latency when a burst of requests arrives. To tackle the identified issues, we propose and develop LEGO, a proactive GC, currently implemented for Go GC, with different GC triggering policies. To mitigate the interference from GC, we design and develop an elastic incremental GC mechanism that minimizes the GC overhead on mutators.

3 LEGO DESIGN

LEGO consists of two important components, an elastic incremental GC mechanism and an augmented Go GC scheduler. Figure 7 shows its design structure. The elastic incremental GC mechanism consists of an incremental GC mechanism and an elastic GC controller. The GC scheduler determines the timing and volume of GC, both before and during a GC period. It also decides which policy is used for GC triggering. The elastic incremental GC mechanism receives signals from the GC scheduler and executes the adaptive GC process.

3.1 Incremental Go GC

The current Go GC supports full GC only, which is triggered on the whole runtime heap and is unable to be paused or stopped during a GC period. LEGO features incremental Go GC, tailoring for its unique GPM model and memory allocator. To adapt incremental GC into Go, LEGO splits GC and non-GC objects. Go uses TCmalloc for memory allocation and allocates objects at the thread level. LEGO makes use of it to depart GC into the P level to minimize the modification in Go runtime. Without rescheduling of Goroutines, all Goruoutines in one P share the same P-level cache, *mspan*. LEGO triggers GC on P-level to minimize the number of cross-region references. However, Go scheduler reschedules Gs into idle Ps for load balancing, which asks for a specific solution to maintaining the validity of reference tracing during GC. Section 4 further illustrates how LEGO tackles the Goroutine immigration issue.

3.2 Elastic GC Controller

LEGO introduces the elastic GC controller for two goals, 1) using the incremental GC mechanism that executes controllable GCs within a shorter time period and involves less resource interference, 2) dynamically determining the volume of GC to be triggered in each GC cycle. The second goal is crucial in tackling bursty requests. The elastic incremental GC grants LEGO the capability to adaptively execute GC based on the system runtime characteristics. It can shorten and stop an ongoing GC when a burst of requests arrives, and execute a full GC when CPU resource usage is not intense.

However, the state-of-the-art GC marking algorithm, tri-color, is unable to dynamically determine the volume of incremental GC during a GC period. The tri-color marking algorithm can only distinguish *marked*, *unmarked*, and *ongoing* objects during a GC period. The two states, *scannable* (all objects involved in the chosen heap area during a GC period) and *unscannable* (vice versa), are pre-fixed in all state-of-the-art GCs. It is impossible to switch the state of *unscannable* objects to the state of *scannable* objects. To adaptively decide the volume of GC during a GC period, LEGO extends the tri-color marking algorithm with the fourth color.

3.3 Quad-Color Algorithm

To support the elastic incremental GC mechanism, we introduce a new Quad-Color algorithm during GC mark phase. It is used for the elastic GC controller. Figure 8 shows the Quad-Color state transform workflow.

We introduce several terminologies. $\sim T$ is the collection of Ps not included in the current *scanned* or *scanning* Ps, while T is the collection of Ps included in the current *scanned* or *scanning* Ps. Set $\{\sim T + T\}$ contains all Ps of the application. When GC starts, GC controller picks Ps from the P Queue, $\sim T$. All Ps in the queue are ranked by the GC scheduler, the earlier with the higher priority. LEGO introduces the fourth color, light gray, extending on the original tri-color algorithm (white, gray, and black). The light gray color is used for objects which should not be deleted when a GC period is fully finished or stopped ahead of time.

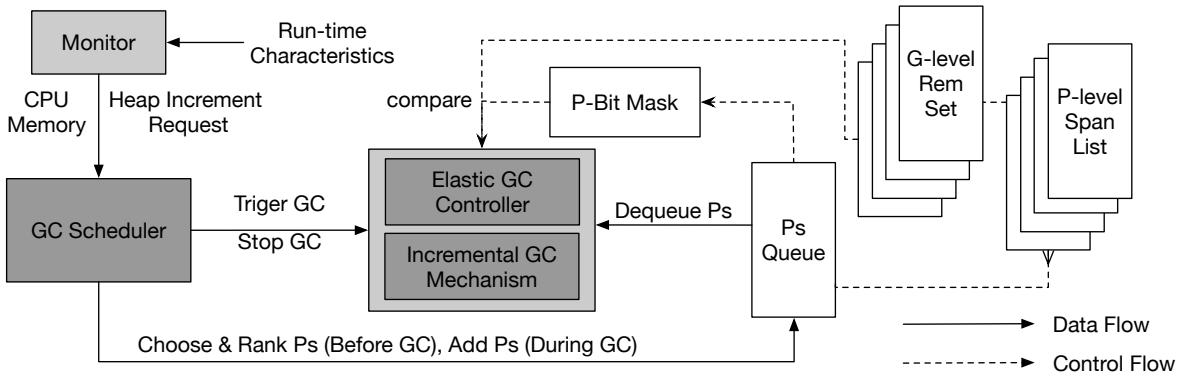


Figure 7: The LEGO design structure. (Gray: Modules; White: Data Structures)

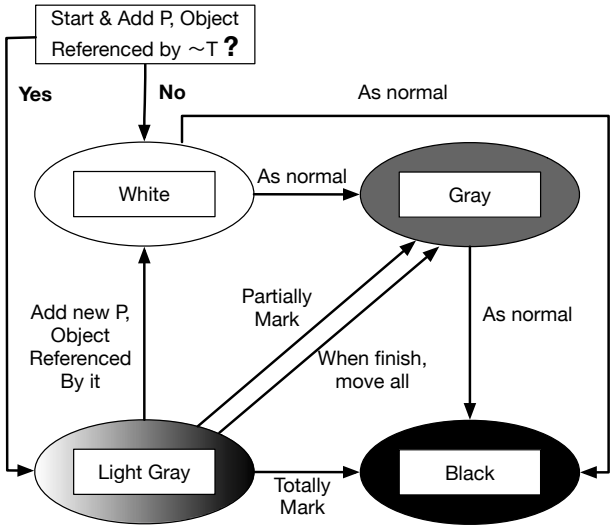


Figure 8: Quad-Color state transform workflow.

When new Ps are added to T each time, LEGO does a comparison with the previous T for objects in the Ps. To do the comparison, remember sets are used for recording cross-P references. The process is described in Section 4. Objects are colored light gray only if they are referenced by Ps in $\sim T$. Following the policies below, light-grayed objects may be transformed into different colors.

- white: an object is only referenced by new T after new Ps are added.
- black: an object is marked and all its references are marked.
- gray: an object is marked but its references are not marked, or the current mark phase is finished.

If an object is not referenced by the current T, it is still uncertain whether it might be used by $\sim T$. The elastic GC controller fades a light-grayed object to a white one only when the object is not referenced by $\sim T$. The fourth color empowers LEGO with the capability to dynamically terminate a GC period for the LC application.

3.4 GC scheduler

The GC scheduler fetches runtime characteristics from the monitor, including the CPU usage, heap usage, and heap increment ratio of the application. Based on the runtime characteristics, the scheduler determines the timing and the volume of GC triggering. We consider three scheduling policies.

- A - Earliest First. The GC scheduler triggers GC as early as possible when CPU usage is below a threshold. Policy A increases the probability of triggering GC at low CPU usages, which can be favorable for a LC application with bursty requests. However, the policy may also lead to a high GC frequency when the application is under low workload. To reduce the frequency of unnecessary GCs, a minimum heap size threshold is introduced for GC triggering.
- B - Default Augmented. The GC scheduler triggers GC when the heap size reaches a similar threshold as in the default Go GC. It makes sure that the threshold can avoid OOM errors. This policy compromises the CPU usage on GC and the tail latency of the application.
- C - Predictive. GC scheduler uses ARIMA [34] model to predict the input pattern and trigger GC under low request pressure. With correct predictions, this policy can offer the best performance among all three policies. However, the policy also requires training datasets in prior for each running application.

GC volume determination. The coordination of GC volume with the quad-color algorithm is achieved by monitoring runtime CPU usage during GC and limiting the number of Ps accordingly. This ensures that the CPU is not fully occupied by GC threads with the assigned Ps. Furthermore, the GC scheduler monitors runtime CPU usage to determine whether to add more Ps into T during the current GC period. In the first two policies, the scheduler naively references runtime CPU usage to make decisions. The threshold varies depending on different applications and system setups, so we add a parameter for user customization. In the experimental

setup, we empirically set the CPU usage threshold at 40%, as this is the average CPU usage when the testbed server reaches its maximum throughput. A higher threshold may result in more CPU interference on mutators.

In Predictive policy, LEGO leverages prediction to select the most suitable time for GC triggering. The GC scheduler directs LEGO to determine how many Ps to execute while maintaining low CPU interference. With the reliable workload prediction of ARIMA, LEGO is more likely to trigger full GC instead of incremental GC. Using full GC for memory reclaim is more effective when the CPU resource is abundant, while using incremental GC is complementary to the full GC under high CPU utilizations.

GC triggering timing. The default full GC executes faster than the incremental GC, as it needs less metadata and scans all Ps at the same time. Thus, the threshold of GC triggering in the default Go GC is the baseline, i.e., latest GC triggering. Under this condition, LEGO switches back to full GC to avoid OOM errors. In LEGO, the concurrent marking time is prolonged while the CPU usage is decreased. When LEGO faces bursty requests and GC is unavoidable, it triggers GC at the lowest level, i.e., one P at a time. Under this condition, we need to ensure the memory is not over the limit. The time to the full GC threshold depends on the number of Ps in the application. LEGO calculates the average time consumption on each P's marking, and multiplies it with the number of Ps to get the latest time interval ahead of the default full GC for triggering the elastic incremental GC mechanism.

Ps enqueue rule. The GC scheduler is also responsible for the ranking of Ps. Here, LEGO uses the least recently scanned list to store all Ps. With the list, LEGO ensures that all Ps have equal opportunity to be chosen during the execution.

4 IMPLEMENTATION

We implement LEGO upon Golang 1.16.6 on Ubuntu 21.04. Here, we discuss the important data structures and illustrate how to address the Goroutine immigration issue.

The elastic incremental GC mechanism asks for delicate information of cross references during runtime. Cross references record the references between different Gs or even between different Ps. We change write barriers in Go to keep the records of these updates. Three levels of metadata are introduced, G, P, and heap levels. G-level remember sets record all references from the source G out of the current G. P-level masks record how many Ps are referenced by the current P. There are also objects that are moved from the G stack and private cache to the universal heap. We add a function to record the relationship between G and P.

Remember set. Each cross reference is at G level or the heap. A remember set records the reference from its destination to the source, which is necessary for incremental GC execution and Goroutine immigration. LEGO records the references by changing the default write barrier and adding entries in the remember set. To distinguish different types of objects in Figure 8, LEGO introduces remember sets at both G level and the universal heap. For G level, it records the cross references from objects in other Gs to the objects in the current G. The heap level remember set records all the objects that escaped from Gs. With the remember set, LEGO can categorize objects into different states during GC. Meanwhile, as Go scheduler reschedules Gs for load balancing, the remember set is also necessary to be used for maintaining the cross reference tracing logics consistently.

Bit mask. Since LEGO triggers GC in the P-level manner, we deploy a P-level mask to reduce the overhead of comparison when new Ps are added during GC runtime. The mask is made of two parts, a reference mask and a reference counting table. P-level bit mask records the cross-reference from the other Ps. Each bit in the mask represents the corresponding P, which owns at least one Goroutine where its objects have references pointing to the current P. The table records the number of Gs referenced by those Ps, which are used to address Goroutine immigration issue. The Reference mask is a bit mask that only paints the bit of reference source granularity.

Goroutine immigration. For load balancing, the Goroutine scheduler schedules *ready* state Goroutines from busy Ps to idle Ps. For example, a Goroutine G1 immigrates from P1 to P2. All existing objects in the immigrated Goroutine are allocated in P1's *mspan* MS1, and new objects will still be allocated on MS1 if there are empty slots. As a result, a *mspan* from P1 is shared by G1, while G1 belongs to P2. If P2 is included in T while P1 is not, we cannot recycle the currently unreferenced objects on MS1, as they might be referenced by objects in P1. Without an effective way, Goroutine immigration would corrupt the incremental GC logic and lead to memory-leak issues.

A direct record of all cross-references will be a catastrophe when new Ps are added into T. Thus, LEGO employs P-level bit masks with the reference counting table. By comparing the bit masks with the current T, LEGO categorizes the objects into four states based on the quad-color state transform workflow, as illustrated in Fig 8. The reference counting table is used when a Goroutine is removed from its current P. Then, all related reference counting numbers are reduced by 1. If some value reaches zero, the corresponding mask bit is cleaned. This is processed during Goroutine immigration, which mitigates its impact on GC performance.

5 EVALUATION

5.1 Setup

Testbed. We use two servers in the evaluation. Each server is equipped with two Intel(R) Xeon(R) Gold 6143 CPUs @ 2.80GHz, 64 GB DRAM, and 1TB HDD.

LC applications. We use four databases developed in Go as LC applications, Badger [4], Bolt [22], Rosedb [8], and TiDB [9]. Badger, an embedded and fast key-value store, is designed for high performance in both reads and writes. Badger is implemented by log-structured merge-tree (LSM Tree). Bolt provides a simple, fast, and reliable database for projects that do not require a full database server. Bolt is implemented by B+ Tree. Rosedb is a fast, reliable, and embedded NoSQL database similar to Rocksdb. Note that Badger, Bolt, and RoseDB are key-value stores. We choose TiDB for SQL evaluation. TiDB is an open-source distributed SQL database that supports Hybrid transactional and analytical processing workloads. It is MySQL compatible and provides strong consistency and high availability. Four databases are running on Ubuntu 21.04. The heap size of each database is set to default 100% unless defined differently.

Microbenchmark. We first evaluate LEGO on LC applications with a microbenchmark. We use Poisson distribution tool FIO [2] to randomly generate input volumes per second with the mean value equal to the maximum server capacity. We use two CPUs for evaluation. The startup heap size is set to 2 GB. Unless mentioned, the Poisson request pattern is used throughout the whole evaluation.
Benchmarks. We evaluate LEGO using Badger, Bolt and TiDB on benchmark Go-YCSB [3]. Note Rosedb is not supported by Go-YCSB, which is a Go port of YCSB [17]. YCSB is an open-source specification and program suite, which is specifically used to compare the relative performance of NoSQL database systems. Go-YCSB fully supports all YCSB generators and the core workload so we can do the basic benchmarks with Go. We choose 50% READ and 50% UPDATE queries in the evaluation, which is a typical setup used in [30]. To evaluate the bursty request issue, we control the *target* value of requests per second. Dynamic *target* is not supported in Go-YCSB, so we add it in Go-YCSB. We use 30 CPUs for evaluation, and the startup heap size is set to 30GB.
Approaches. We use three approaches in performance evaluation, 1) the default Go GC (noted as Go GC), 2) a passive incremental version of LEGO (noted as passiveLEGO), which emulates the production G1 GC used in JVM, 3) and LEGO. Unless mentioned, LEGO uses policy B, Default Augmented, in the GC Scheduler.
Metrics. In primary, we measure the tail latency and throughput of the LC applications. We also form a metric to study the relationship between GC triggering and bursty conditions. We record the sum of the number of GCs triggered under high CPU usage scenarios (higher than the threshold) and under low CPU usage scenarios (lower than the threshold). The default maximum RPS of the server is set to be the threshold that can maintain the SLO of LC applications. The metric, called $Ratio_{Burst}$ (R_B), is the ratio of GC triggered under high CPU usage scenarios and under low CPU usage scenarios, which is multiplied by the proportion of bursty requests in the total. A high R_B means more proportions of GCs are triggered under high CPU usages. We use a metric of GC burstiness to study the benefit of GC triggering under low CPU usages.

In the following, we first evaluate the tail latency and the throughput of the three LC applications. We also compare the passive GC mechanism with LEGO, and discuss different scheduling policies used in LEGO GC scheduler. We then conduct sensitivity analysis by examining the performance of LEGO under different memory pressure levels and heap sizes. Finally, we discuss the overhead of LEGO.

5.2 The Tail Latency

We evaluate the tail latency of the LC applications with both the microbenchmark and Go-YCSB benchmark. Experiments are conducted in two configurations, 1) LC applications using LEGO, and 2) LC applications using the default Go GC. Under each configuration, we run the experiment for two hours and measure the average values as well as the real-time values during GC runtime. Due to the different implementations and CPU usages of the three LC applications, their RPS are different in meeting the maximum server capacity.

Table 3 shows all results under the five configurations. We can observe that LEGO renders the lowest tail latency in average as well as during GC runtime for all five configurations. Compared with the default Go GC, for the microbenchmark LEGO can reduce the p99 (p99.9) latency by 13.3% (33.3%), 18.1% (40.3%), 12.7%(25.6%), and 21.6% (33.8%) for Badger, Bolt, TiDB, and Rosedb, respectively. For Go-YCSB, LEGO can reduce the p99 (p99.9) latency by 35.6% (36.7%), 19.7% (23.4%), and 9.5% (17.3%) for Badger, Bolt, and TiDB respectively. Badger uses LSM tree, which speeds up write operations while leading to multiple read operations in reading a modified entry. Bolt employs B+ Tree to strengthen its capability in concurrent readings. The trade-off between write and read operations makes Badger more memory consuming. Thus, compared with Bolt, Badger uses more memory and triggers more GCs in face of the same number of incoming requests. Due to more rescheduled GCs, LEGO obtains more significant improvement for Badger than for Bolt.

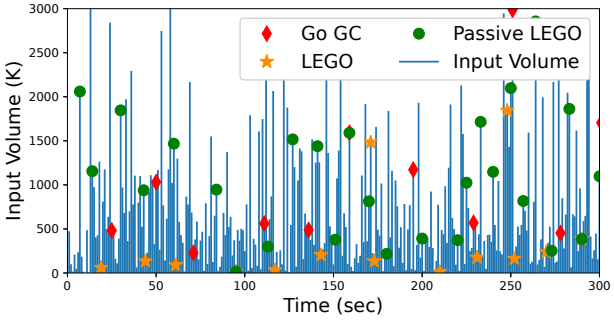


Figure 9: GC triggering of the three approaches.

Among the LC applications, LEGO achieves relatively less improvement for TiDB due to its storage engine optimization. TiKV serves as the underlying storage engine for TiDB, utilizing RocksDB for data storage and management. TiKV supports multi-version concurrency control, which allows multiple versions of the same row to be stored in RocksDB. To reduce resource usage, TiKV records the concurrency control statistics. The optimization allows TiDB to check the table properties before examining each region row by row, resulting in fewer memory objects generated in the runtime heap and reducing the GC overhead for memory reclaim.

We also evaluate the tail latency during GC runtime. Under the seven configurations, compared with Go GC, LEGO reduces the p95 (p99) latency during GC runtime by 54.8% (62.3%), 65.7% (70.1%), 36.4% (56.8%), 61.5% (70.3%), 71.9% (81.8%), 75.8% (81.2%), 70.7% (82.9%), respectively. Note even with LEGO, the tail latency can still be non-trivial during GC runtime, as the GC process owns higher priority compared to the mutators.

5.3 Passive GC v.s. Proactive GC

We now investigate the performance of using LEGO without its GC scheduler (noted as passive LEGO) to emulate G1 GC, a production concurrent GC that is the default GC on JDK 9. We choose G1 GC instead of more recently proposed JVM GCs for performance comparison because a recent study finds those new GC mechanisms indeed incur higher tail latency [51] than G1 GC does. Passive LEGO is set to trigger GC with 50% volume of the runtime heap. We use Badger with Go-YCSB benchmark in the performance evaluation.

Table 3: The tail latency (ms) and throughput of LC applications with the microbenchmark and GO-YCSB.

Benchmarks	LC Applications	GC types	Average Application Latency and Throughput				Latency during GC Runtime	
			p99	p99.9	Average	Throughput (RPS)	p95	p99
Microbench	Badger	GoGC	12.1	19.8	6.7	50K/s	42	114
		LEGO	10.5	13.2	6.8	50K/s	18	43
	Bolt	GoGC	18.4	31.5	9.3	60K/s	67	216
		LEGO	14.9	18.8	9.5	60K/s	23	65
	TiDB	GoGC	8.7	17.2	5.3	50K/s	33	95
		LEGO	7.6	12.8	5.4	50K/s	21	41
	Rosedb	GoGC	22.7	37.0	11.5	50K/s	83	263
		LEGO	17.8	24.5	11.7	50K/s	32	78
Go-YCSB	Badger	GoGC	28.4	38.7	12.7	750K/s	124	453
		LEGO	18.3	24.5	12.9	750K/s	35	87
	Bolt	GoGC	21.9	27.4	10.9	900K/s	128	350
		LEGO	17.6	21.0	11.0	900K/s	31	69
	TiDB	GoGC	10.6	23.1	7.3	750K/s	92	188
		LEGO	9.0	19.1	7.5	750K/s	27	51

Figure 9 shows the input volume of Badger in RPS in five minutes of experimentation. Three types of points show the timing and the input volume when GC is triggered due to the three approaches. Go GC and LEGO incur similar numbers of GC triggering, while the passive LEGO incurs about 2x of GC triggering because it cannot dynamically adjust the GC volume at runtime. When the volume is set at one half of the heap, the times of GC triggering increase dramatically. Without an adaptive GC scheduler, the passive GC tends to trigger GC under high input volumes, similarly to Go GC. On the other hand, LEGO triggers the majority of GCs under low input volumes (i.e. under low CPU usages). There are two exceptions occurring at 171s and 248s, when LEGO is triggered under bursty conditions. Two following GCs are triggered at 173s and 252s. During the first 2 GCs, LEGO limits the GC volume for low latency, and later triggers the two extra GCs when the CPU usage is low.

Table 4 gives the experimental statistics in the number of GC triggering, the average input volume at GC triggering, R_B, and p99 tail latency. The p99 tail latency due to LEGO, passive LEGO and Go GC are 17.9ms, 31.2ms, and 27.3ms, respectively. Passive LEGO yields the worst outcome. Due to the extra data recording, a naive incremental GC (passive LEGO) increases the resource usage and reduces the memory reclaim volume in each GC cycle. Without a proactive GC scheduler, passive LEGO incurs 14% longer p99 tail latency. LEGO triggers slightly more GCs than Go GC does, while its R_B and the average input volume at GC triggering are dramatically smaller than those of the other two approaches. A smaller R_B means fewer GC triggered under high CPU usages. Compared with Go GC and the tailored G1 GC, LEGO improves the p99 tail latency by 34.5% and 42.7% respectively.

Overall, LEGO demonstrates significant improvement over both Go GC and G1 GC (emulated by passive LEGO).

Table 4: Statistics of GC triggering of three approaches.

	LEGO	Passive LEGO	Go GC
Number of GC triggering	14	26	11
Average Input at GC(K)	355	1104	1334
R_B	0.331	0.955	1.031
p99(ms)	17.9	31.2	27.3

Table 5: Statistics of three policies under two RPS values.

Input Volume	Statistics	Policies		
		A	B	C
750K RPS /sec	GC triggering times	360	156	146
	Avg Input at GC (K)	193	341	289
	R_B	0.072	0.310	0.181
	p99(ms)	18.1	17.7	17.5
1500 RPS /sec	GC triggering times	835	340	311
	Avg Input at GC (K)	470	683	552
	R_B	0.372	1.396	1.023
	p99(ms)	47.8	54.2	43.3

5.4 Application Throughput

We run LC applications Badger (Bolt) with varying RPS, ranging from 500K to 1M (1.3M) with 50K (100K) as a step. We study the application throughput due to the three approaches. We collect the p99 tail latency at each RPS and study how LEGO improves the application throughput, compared with the default Go GC and the passive LEGO.

Figure 10 shows the relationship between the p99 tail latency and the application throughput (RPS) of the three approaches. The red line shows the p99 tail latency of 28.4ms (21.9ms), when the default Go GC runs at the maximum server capacity at 750K (900K) RPS. Due to the overhead of the incremental GC, the passive LEGO reaches a similar p99 tail latency when RPS is decreased to 700K (800K). All three approaches lead to increasing application throughput as more requests are processed per second. Without rescheduling the GC process, the p99 tail latency of Go GC increases much faster than LEGO does. LEGO reaches a similar p99 tail latency of 29.1ms (21.6ms), when the RPS is increased to 1,000K (1,100K) RPS for Badger (Bolt). Thus, compared with the default Go GC and passive LEGO, LEGO improves the throughput of Badger (Bolt) by 33% (22%) and 42% (38%), respectively, under similar tail latency.

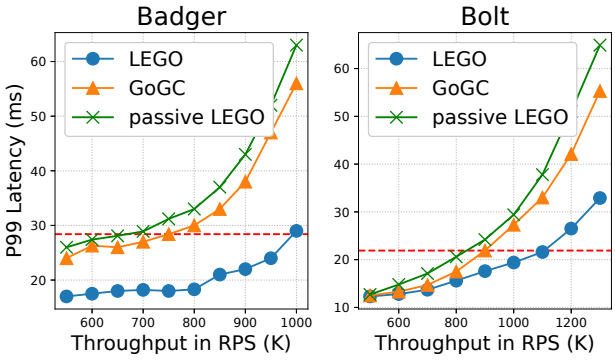


Figure 10: Application throughput of three approaches.

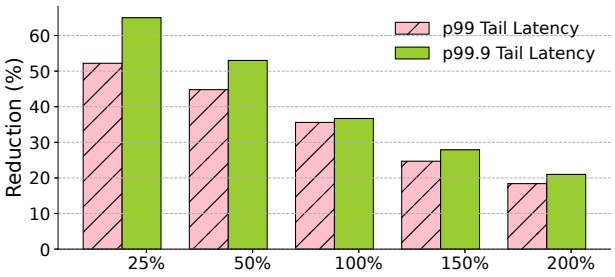


Figure 11: Latency reduction with different heap sizes.

5.5 Scheduling Policies

We examine the three policies of LEGO GC scheduler, Earliest First (A), Default Augmented (B), and Predictive (C) under two input conditions, 750K PRS as the normal traffic condition and 1,500K RPS as the heavy condition. The heap size thresholds of A are set at 50% and 30% under the normal and heavy conditions, respectively. C is trained by ARIMA in a 10-hour input RPS record and predicts the next 1-hour RPS.

Table 5 shows policy C outperforms the two other policies, with 17.5ms and 43.3ms in the p99 tail latency under normal and heavy-duty conditions, respectively. Compared with A and B in normal (heavy) conditions, C reduces the tail latency by 3.4% (9.5%) and 1.2% (20.2%), respectively. Accurate prediction of incoming traffic volume enables C to trigger the least times of GC. It avoids the two extra GC triggering due to Policy B (at 171s and 248s in Figure 9). It also increases the opportunity in triggering GC under low CPU usages. The performance difference due to policies A and B depends on the incoming traffic conditions. Under normal conditions, B obtains shorter p99 tail latency (17.7ms) than A does (18.1ms). Though A has the lowest R_B among all three approaches, the frequency of GC triggering is so high that incurs significant CPU interference. Under heavy traffic conditions, A obtains smaller p99 tail latency (47.8ms) than B does (54.2ms). A triggers GC as early as possible when both thresholds of heap size and CPU usage are met. It yields the higher possibility in triggering GC under low CPU usages.

Overall, Policy C yields the best performance with correct predictions. Policy B fits under a normal traffic condition, while Policy A works better under heavy traffic conditions.

5.6 Sensitivity of Memory Pressure

We now examine LEGO under different memory pressures. We run Badger on Go-YCSB with different heap sizes to evaluate the latency reduction by LEGO compared to the default Go GC. Figure 11 shows the percentile of the tail latency reduction when the size of the runtime heap is set to 25%, 50%, 100%, 150%, and 200%, respectively. We can see that LEGO obtains just marginal improvement for the p99 and p99.9 tail latency under 150% and 200% heap sizes. However, as the heap size decreases, the latency reduction due to LEGO improves significantly. The rationale is that there are more frequent GC triggering with small heap sizes, which in return increases the CPU interference of GC and mutators. LEGO yields higher possibility in triggering GC under low CPU usages, which mitigates the CPU contention with mutators. As a result, LEGO significantly reduces the tail latency. For example, when the heap size is set at 50%, the latency reductions for p99 and p99.9 latency due to LEGO are 46.2% and 53.4%, respectively.

Table 6: Statistics in multi-tenant environments.

	LC p99 Tail Latency (ms)	Batch Jobs (# per hour)	Ave. CPU usage at GC triggering
LEGO	25.9	175	26%
Go GC	32.0	172	47%

5.7 Multi-tenancy

We conduct performance evaluation of LEGO under multi-tenancy. We use Badger as the LC application and Kmeans from Hibench running on Spark as the batch jobs. We use 20 logical CPUs (10 physical cores with Hyperthreading) for the experiment, in which 10 logical CPUs are assigned to Badger and the other 10 paired logical CPUs are assigned to the batch job pool. Thus, there are CPU interference between Badger and batch jobs. Table 6 presents the p99 tail latency and the average CPU usage when GC is triggered. The results show that LEGO reduces the p99 tail latency by 19.1% compared to the default Go GC. Furthermore, the average CPU usage when GC is triggered with LEGO is 44% lower, reducing CPU interference from GC threads. LEGO also marginally increases the throughput of batch jobs, as less CPU preemption occurs during GC. Note that resource scheduling and performance isolation between different applications is out of scope of this study.

5.8 Overhead

LEGO introduces the elastic incremental GC mechanism that dynamically determines the GC volume at runtime. Compared with the default Go GC, LEGO employs an always-on write barrier for object recording, which increases the average request latency by up to 2%. LEGO also increases the basic heap usage due to the extra data structure for cross-references, which implicitly increases the frequency of GC. However, the benefits of the elastic incremental GC mechanism and adaptive GC scheduler significantly outperform the overhead. Note that the number of Ps is counted by the default Go runtime. LEGO does not introduce overhead in this regard.

6 RELATED WORK

LC Application QoS. Many studies focus on meeting the SLO of LC applications from the perspective of reducing the tail latency [11, 23, 25, 39, 52]. For example, PerfIso [28] is on CPU isolation for CPU sharing between LC applications and batch jobs. FlatStore [16] is a KV store that uses logs in persistent memory for serving data requests. Caladan [25] proposes a new CPU scheduler in the Linux kernel that improves QoS of LC applications, especially the tail latency. Perséphone [23] is a novel kernel-bypass OS scheduler designed to minimize the tail latency for applications executing at the microsecond scale. LEGO tackles the GC interference on LC applications, which is orthogonal to those studies.

GC Optimization. Most studies on GC are focused on JVM [12, 24, 27, 27, 33, 36–38, 46, 49–51]. LXR GC [51] demonstrates the production G1 GC yields smaller tail latency for LC applications, compared with the concurrent GC mechanisms such as ZGC GC [20] and Shenandoah GC [19]. Though G1 GC incurs more STW pauses, the overall tail latency is indeed reduced. Yak [37] develops a GC algorithm for distributed data processing systems. ROLP [12] profiles object lifetime and uses the profiled information for efficient GC. iGC [50] introduces a proactive GC to reduce the tail latency in cloud environments. Different from the most tracing GCs, LRX GC [51] evolves the original reference counter based GC and achieves significant improvement on LC application QoS. However, the mechanism does not apply for Go, because Go GC is neither generational nor copy-and-compact. Platinum [47] leverages idle cores during STW pauses and grants them to mutators for utilizing idle computing resources. LEGO focuses on Go GC, while its adaptive incremental GC mechanism can also be extended and deployed in other automatic memory management programming languages.

GC in New Systems. There are GC studies focusing on new systems, such as non-volatile memory (NVM) and far memory systems. For example, Semeru [44] provides a distributed GC that offloads object tracing to memory servers so that tracing is performed close to data in memory-disaggregated systems. Work [48] optimizes GC atop NVM systems by introducing two NVM-aware optimizations, write cache and header map, to tackle the bandwidth limitation of NVM. Memliner [45] improves the performance of far-memory systems by lining up memory accesses, which alleviates the gap between the application QoS and the GC.

Performance Interference and Isolation. Prior works have intensively tackled performance interference in several ways. Early approaches disallow LC applications from sharing resources with other applications to avoid performance interference [13–15, 31, 39, 40, 42, 43]. Other early approaches avoid co-scheduling applications that are likely to interfere with each other [21, 35]. The third category of research focuses on eliminating interference altogether, by partitioning and isolating resources among co-scheduled services with feedback control based techniques [22, 29, 31, 39]. LEGO tackles the interference between GC threads and mutators of LC applications, which is complementary to the existing studies.

7 CONCLUSION

LEGO is motivated by severe performance degradation due to the CPU interference between concurrent GC threads and mutators of LC applications during GC runtime in face of bursty requests. We find that proactively scheduling GC threads under low CPU usages opens up a significant opportunity for application performance improvement. LEGO designs the adaptive incremental GC mechanism for the first time. We develop LEGO in Go, as the current GC mechanisms for JVM are ineffective for Go due to its unique GPM thread scheduling and memory allocation model. Experimental results show that compared to the default Go GC and a tailored G1 GC, LEGO significantly improves both the tail latency and the throughput for LC applications developed in Go.

In future, we plan to extend LEGO to other programming languages that deploy automatic memory management.

REFERENCES

[1] 2020. GO GC problem in Discord. https://discord.com/blog/why-discord-is-switching-from-go-to-rust.
[2] 2022. Flexible I/O tester. https://github.com/axboe/fio.
[3] 2022. Go-YCSB. https://github.com/pingcap/go-ycsb.
[4] 2023. Badger. https://pkg.go.dev/github.com/dgraph-io/badger.
[5] 2023. Beego. https://github.com/beego/beego.
[6] 2023. Gin Web Framework. https://gin-gonic.com/.
[7] 2023. Go Case Studies. https://go.dev/solutions/#case-studies.
[8] 2023. Rosedb. https://github.com/roseduan/rosedb.
[9] 2023. TiDB: Open, Unified, Distributed SQL. https://www.pingcap.com/.
[10] 2023. Tri Color Marking. https://en.wikipedia.org/wiki/Tracing_garbage_collection#Tri-color_marking.
[11] Daniel S Berger, Benjamin Berg, Timothy Zhu, Siddhartha Sen, and Mor Harchol-Balter. 2018. RobinHood: Tail Latency Aware Caching–Dynamic Reallocation from Cache-Rich to Cache-Poor. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). Oct. 8–10, 2018, CARLSBAD, CA, USA.*
[12] Rodrigo Bruno, Duarte Patricio, José Simão, Luis Veiga, and Paulo Ferreira. 2019. Runtime object lifetime profiler for latency sensitive big data applications. In *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys). March 25-28, 2019, Dresden Germany.*
[13] Wei Chen, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2019. Os-augmented oversubscription of opportunistic memory with a user-assisted oom killer. In *Proceedings of the 20th International Middleware Conference (Middleware). Dec. 9-13, 2019, Davis CA USA.*
[14] Wei Chen, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2019. Pufferfish: Container-driven elastic memory management for data-intensive applications. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC). Nov. 20-23, 2019, Santa Cruz CA USA.*
[15] Wei Chen, Jia Rao, and Xiaobo Zhou. 2017. Preemptive, low latency datacenter scheduling via lightweight virtualization. In *2017 USENIX Annual Technical Conference (USENIX ATC 17). July 12–14, 2017, Santa Clara, CA.*
[16] Youmin Chen, Youyou Lu, Fan Yang, Qing Wang, Yang Wang, and Jiwu Shu. 2020. Flatstore: An efficient log-structured key-value storage engine for persistent memory. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). March 16-20, 2020, Lausanne, Switzerland.*
[17] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing (SoCC). June 10-11, 2010, Indianapolis Indiana USA.*
[18] Oracle Corperation. 2016. Java Garbage Collection Stop The World Event. https://www.oracle.com/technetwork/tutorials/tutorials-1873457.html
[19] Oracle Corporation. 2022. The Shenandoah Garbage Collector. https://wiki.openjdk.org/display/shenandoah/Main
[20] Oracle Corporation. 2022. The Z Garbage Collector. https://openjdk.java.net/projects/zgc/
[21] Christina Delimitrou and Christos Kozyrakis. 2013. Paragon: QoS-aware scheduling for heterogeneous datacenters. *ACM SIGPLAN Notices* (2013).
[22] Christina Delimitrou and Christos Kozyrakis. 2017. Bolt: I know what you did last summer... in the cloud. *ACM SIGARCH Computer Architecture News* (2017).
[23] Henri Maxime Demoulin, Joshua Fried, Isaac Pedisich, Marios Kogias, Boon Thau Loo, Linh Thi Xuan Phan, and Irene Zhang. 2021. When Idling is Ideal: Optimizing Tail-Latency for Heavy-Tailed Datacenter Workloads with Perséphone. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP). Oct. 26-29, 2021, Virtual Event Germany.*
[24] David Detlefs, Christine Flood, Steve Heller, and Tony Printezis. 2004. Garbage-first garbage collection. In *Proceedings of the 4th international symposium on Memory management (ISMM). Oct. 24-25, 2004, Vancouver BC Canada.*

[25] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2020. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). Nov. 4-6, 2020, virtual.*

[26] Sanjay Ghemawat and Paul Menage. 2005. TCMalloc: Thread-caching malloc. http://goog-perftools.sourceforge.net/doc/tcmalloc.html.

[27] Ionel Gog, Jana Giceva, Malte Schwarzkopf, Kapil Vaswani, Dimitrios Vytiniotis, Ganesan Ramalingam, Manuel Costa, Derek G Murray, Steven Hand, and Michael Isard. 2015. Broom: Sweeping out garbage collection from big data systems. In *15th Workshop on Hot Topics in Operating Systems (HotOS XV). May 18-20, 2015, Kartause Ittingen, Switzerland.*

[28] Călin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, et al. 2018. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *2018 USENIX Annual Technical Conference (USENIX ATC 18). July 11–13, 2018, Boston, MA, USA*.

[29] Harshad Kasture, Davide B Bartolini, Nathan Beckmann, and Daniel Sanchez. 2015. Rubik: Fast analytical power management for latency-critical systems. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). Dec. 5-12, 2015, Waikiki, HI, USA*. IEEE.

[30] Harshad Kasture and Daniel Sanchez. 2016. Tailbench: a benchmark suite and evaluation methodology for latency-critical applications. In *2016 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 1–10.

[31] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. 2015. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA). June 13-17, 2015, Portland Oregon.*

[32] Lu Lu, Xuanhua Shi, Yongluan Zhou, Xiong Zhang, Hai Jin, Cheng Pei, Ligang He, and Yuanzhen Geng. 2016. Lifetime-Based Memory Management for Distributed Data Processing Systems. *Proc. VLDB Endow.* (2016).

[33] Martin Maas, Tim Harris, Krste Asanović, and John Kubiatowicz. 2015. Trash day: Coordinating garbage collection in distributed systems. In *15th Workshop on Hot Topics in Operating Systems (HotOS XV). May 18-20, 2015, Kartause Ittingen, Switzerland.*

[34] Spyros Makridakis and Michele Hibon. 1997. ARMA models and the Box–Jenkins methodology. *Journal of forecasting* 16, 3 (1997), 147–163.

[35] Jason Mars and Lingjia Tang. [n. d.]. Whare-map: Heterogeneity in" homogeneous" warehouse-scale computers. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA). June 23-27, 2013, Tel-Aviv Israel.*

[36] Kris Mok. 2011. JVM@Taobao. https://www.slideshare.net/RednaxelaFX/jvm-taobao

[37] Khanh Nguyen, Lu Fang, Guoqing Xu, Brian Demsky, Shan Lu, Sanazsadat Alamian, and Onur Mutlu. 2016. Yak: A High-Performance Big-Data-Friendly Garbage Collector. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). Nov. 2–4, 2016, SAVANNAH, GA, USA*.

[38] Khanh Nguyen, Kai Wang, Yingyi Bu, Lu Fang, Jianfei Hu, and Guoqing Xu. 2015. Facade: A compiler and runtime for (almost) object-bounded big data applications. *ACM SIGARCH Computer Architecture News* (2015).

[39] Aidi Pi, Junxian Zhao, Shaoqi Wang, and Xiaobo Zhou. 2021. Memory at your service: fast memory allocation for latency-critical services. In *Proceedings of the 22nd International Middleware Conference (Middleware). Dec. 6-10, 2021, Québec city Canada.*

[40] Aidi Pi, Xiaobo Zhou, and Chengzhong Xu. 2022. Holmes: SMT Interference Diagnosis and CPU Scheduling for Job Co-Location. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing.*

[41] David K. Rensin. 2015. *Kubernetes - Scheduling the Future at Cloud Scale.* http://www.oreilly.com/webops-perf/free/kubernetes.csp

[42] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: Flexible, Scalable Schedulers for Large Compute Clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys). April 15-17, 2013, Prague Czech Republic.*

[43] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-Scale Cluster Management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys). April 21-24, 2015, Bordeaux France.*

[44] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2020. Semeru: A Memory-Disaggregated Managed Runtime. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). Nov. 4-6, 2020, virtual.*

[45] Chenxi Wang, Haoran Ma, Shi Liu, Yifan Qiao, Jonathan Eyolfson, Christian Navasca, Shan Lu, and Guoqing Harry Xu. 2022. {MemLiner}: Lining up Tracing and Application for a {Far-Memory-Friendly} Runtime. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22).*

[46] Jingjing Wang and Magdalena Balazinska. 2017. Elastic memory management for cloud data analytics. In *2017 USENIX Annual Technical Conference (USENIX ATC 17). July 12–14, 2017, Santa Clara, CA.*

[47] Mingyu Wu, Ziming Zhao, Yanfei Yang, Haoyu Li, Haibo Chen, Binyu Zang, Haibing Guan, Sanhong Li, Chuansheng Lu, and Tongbao Zhang. 2020. Platinum: A {CPU-Efficient} Concurrent Garbage Collector for {Tail-Reduction} of Interactive Services. In *2020 USENIX Annual Technical Conference (USENIX ATC 20).*

[48] Yanfei Yang, Mingyu Wu, Haibo Chen, and Binyu Zang. 2021. Bridging the performance gap for copy-based garbage collectors atop non-volatile memory. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys).*

[49] Junxian Zhao, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2021. Flashbyte: Improving memory efficiency with lightweight native storage. In *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid). May 10-13, 2021, Melbourne, Australia.* IEEE.

[50] Junxian Zhao, Aidi Pi, Xiaobo Zhou, Sang-Yoon Chang, and Chengzhong Xu. 2022. Improving Concurrent GC for Latency Critical Services in Multi-tenant Systems. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference (Middleware).*

[51] Wenyu Zhao, Stephen M. Blackburn, and Kathryn S. McKinley. 2022. Low-Latency, High-Throughput Garbage Collection. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)* (San Diego, CA, USA).

[52] Timothy Zhu, Michael A Kozuch, and Mor Harchol-Balter. 2017. Workloadcompactor: Reducing datacenter cost while providing tail latency slo guarantees. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC). Sept. 24-27, 2017, Santa Clara California.*